What I added in PR #8

WIB, DAPHNE and PDU emulators for bridge tests

Manuel Arroyave

10 November 2026

Threads and asyncio handle waiting differently

	Threads	Asyncio
Switching	Operating system	Event loop, using `await`
Slow device	Other threads continue	Other tasks continue
Library calls	Can use blocking calls	Needs async calls
Watch for	Shared data may need locks	A blocking call can stall the loop

A blocking WIB read can hold up the loop until its timeout.
Use an async read or a worker thread.

1. I added three simulated devices

DUNE/dune-dscs, PR #8

WIB

ZeroMQ + protobuf

DAPHNE

ZeroMQ + protobuf v2

PDU

SNMP v2c / Raritan

A bridge can talk to them without the hardware.

2. I used the existing message definitions

WIB and DAPHNE: bundled protobuf definitions.

PDU: Raritan PDU2-MIB outlet commands.

I recorded the source revisions and hashes.

3. I made commands change simulated state

WIB	FEMB power, registers and timing state
DAPHNE	Configuration and trigger thresholds
PDU	Outlet on/off state

A write can be checked with a read.

4. I made each device easy to run

```
uv run python -m dune_dscs.emulator wib
```

Choose `wib`, `daphne` or `pdu`.

Each runs in its own process, using asyncio.

Host and port are configurable; defaults use loopback.

5. I checked the emulators

Local tests and Ignition on dune-fd-test01

Ran 963 devices locally.

Read 627 devices in Ignition.

Checked writes, device loss and recovery.

The Ignition test used a separate adapter, outside PR #8.

Hardware and crate IPMI still need testing.

6. Geoff can use them to test his bridge

Point the bridge’s device clients at the emulators.

Publish the results over OPC UA to Ignition.

Start with one device, then increase the fleet.

PR #8 supplies the emulators. The new bridge is still to be written.